Table of Contents

1 Introduction....1

RaspberryPi3-General Purpose Input Output (GPIO)....1

2 The GPIO pin assignments for Oradio3....2

The pin assignment for current version of the Oradio3....2

The RaspiOS GPIO boot configuration....2

1 The python RPI.GPIO package....2

2 GPIO software interface....2

3 RPI.GPIO configuration programming....3

4 Multi-user/Multi-task considerations for RPI.GPIO....3

5 GPIO Design decision....4

3 Top-down layered static view....5

4 Detailed design of the gpio_service module....7

GPIOService Class....7

1 Purpose....7

2 Design & Threading....8

3 API Reference....8

GPIO Constants....8

Methods:....9

4 Error handling....10

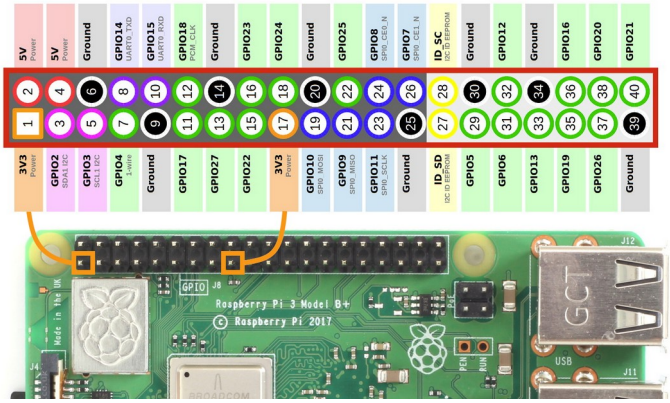
5 Attributes for module test....10

6 Module test....10

1 Introduction

1.1 RaspberryPi3-General Purpose Input Output (GPIO)

All Raspberry Pi models come with a 40 GPIO pins (P1 header) on the main board. It is used for digital I/O, communication protocols, PWM, and interrupts. The GPIO numbers are assigned by the Broadcom (BCM) chip. This design will describe to use of the digital I/O pins for controlling the connected LEDs and getting the state of the connected touch buttons.



The P1 header is connected to the front panel PCB of the Oradio3, on which the led indicators, touch buttons and volume button are located. These items are connected to the P1 header via the front panel PCB.

2 The GPIO pin assignments for Oradio3

2.1 The pin assignment for current version of the Oradio3.

The front panel PCB is connected to the P1 header using following pins.

Pin	Function	Pull-Up/Down	Usage
7	Output	Pull-Up	LED-PRESET3
15	Output	Pull-Up	LED-PLAY
23	Output	Pull-Up	LED-STOP
24	Output	Pull-Up	LED-PRESET1
25	Output	Pull-Up	LED-PRESET2
5	Input	Pull-Up	BUTTON-PRESET2
6	Input	Pull-Up	BUTTON-STOP
9	Input	Pull-Up	BUTTON-PLAY
10	Input	Pull-Up	BUTTON-PRESET3
11	Input	Pull-Up	BUTTON-PRESET

All the IO-pins require a Pull-Up configuration.

2.2 The RaspiOS GPIO boot configuration

In the config file /boot/firmware/config.txt the low-level for the GPIO pins are set. The Raspberry Pi uses Device Tree (DT) overlays to configure the hardware, including GPIO pins. Overlays are applied using dtoverlay, e.g. to configure an I/O for i2c or UART. In our case we can define the outputs for the LEDs as follows in the config.txt:

```
gpio=23=op,dl        # output, driving low (LED-STOP)
gpio=24=op,dh        # output, driving high (LED-PRESET1)
gpio=25=op,dh        # output, driving high (LED-PRESET2)
gpio=7=op,dh         # output, driving high (LED-PRESET3)
gpio=15=op,dh        # output, driving high (LED-PLAY)
```

During boot time these settings will be applied to the output pins. The LED-STOP will be ON during boot-time

2.2.1 The python RPI.GPIO package

2.2.2 GPIO software interface

The package is available as pip package: https://pypi.org/project/Rpi.GPIO/. It provides a simple`` interface for configuring the pins as inputs or outputs, reading their state, and controlling them.

There are two ways of numbering the IO pins on a Raspberry Pi within Rpi.GPIO:

- The first is using the BOARD numbering system. This refers to the pin numbers on the P1 header of the Raspberry Pi board.
- The second numbering system is the BCM numbers. This is a lower level way of working - it refers to the channel numbers on the Broadcom SOC.

It is required to specify which numbering system is used.

A valuable function is GPIO.add_event_detect(). It is a function used in Raspberry Pi programming to set up an event detection on a specific GPIO pin. This allows the program to respond to changes in the pin's state, such as detecting when a button is pressed or released.

2.2.3 `RPI.GPIO configuration programming`

GPIO configuration	Instruction
Specify numbering system	GPIO.setmode(GPIO.BCM)
Input	GPIO.setup(channel, GPIO.IN)
Output	GPIO.setup(channel, GPIO.OUT)
Output with initial value	`GPIO.setup(channel, GPIO.OUT, initial=GPIO.HIGH)`
Pull up resistors	`GPIO.setup(channel, GPIO.IN, pull_up_down=GPIO.PUD_UP)`
Pull down resistors	`GPIO.setup(channel, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)`
Events	`GPIO.add_event_detect(channel, GPIO.RISING, callback=my_callback)`
	`Events can be GPIO.RISING, GPIO.FALLING or GPIO.BOTH.`

2.2.4 `Multi-user/Multi-task considerations for RPI.GPIO`

The GPIO interface can be used by any user or task running on the Rpi3b. So it may require some protection to prevent unexpected results on I/O pins.

The RPI.GPIO library also offers good multi-task protections:

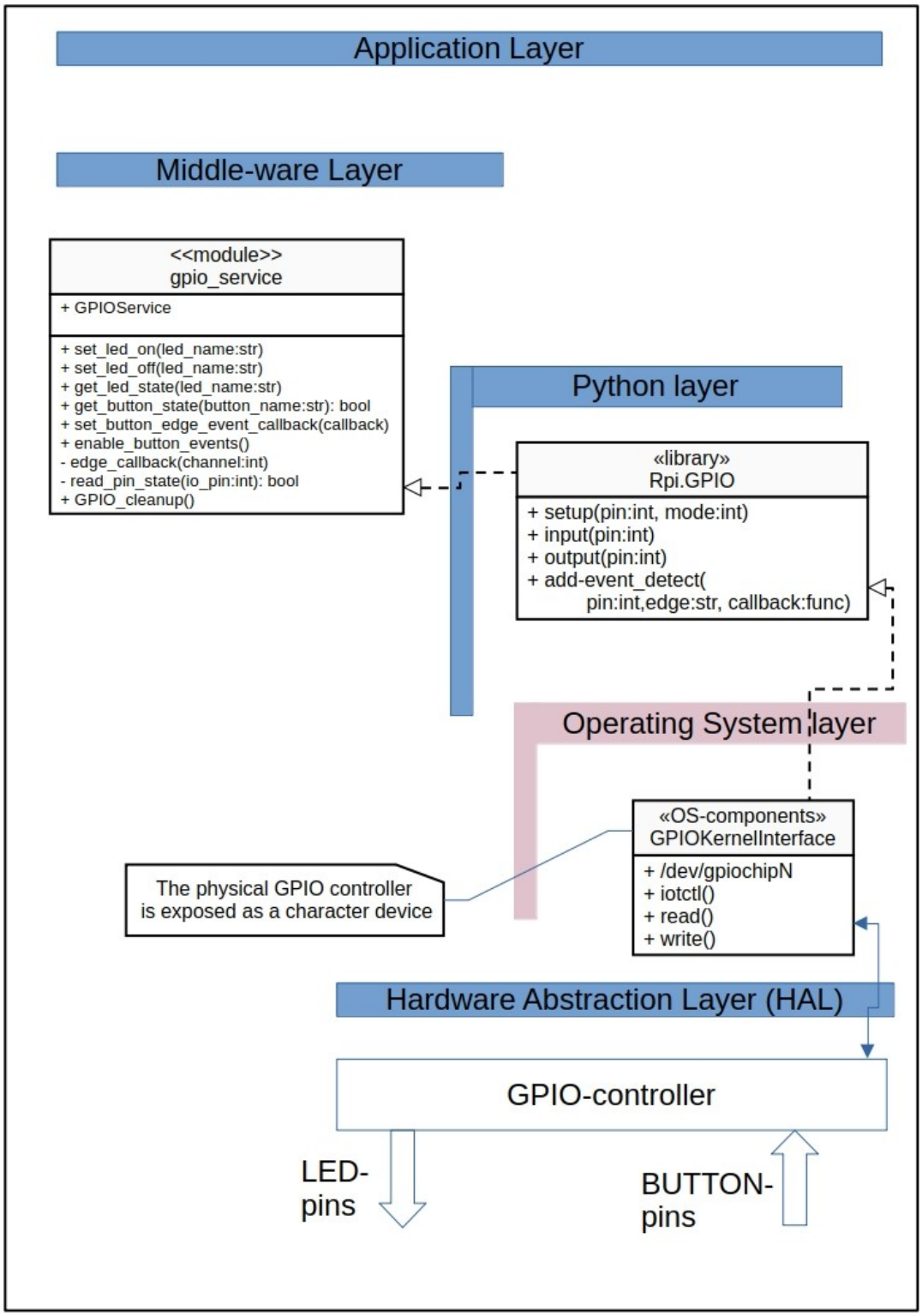
- It offers atomicity on the GPIO writes: a call to `GPIO.output(pin, state)` the library makes a system call to write to either:
  - `/sys/class/gpio/gpioX/value` (sysfs interface), or
  - `/dev/gpiomem` (memory-mapped interface).

  The kernel guarantees that the write operation completes in one uninterruptible step.
- If two programs try to write to the same pin simultaneously, the kernel will serialize the writes. The second write will not interrupt the first—it will simply overwrite the result of the first write as soon as the first operation is done.
- The kernel does **not** stop or preempt a GPIO write operation to handle a new one. Instead, it processes writes in the order they are received (or as scheduled by the kernel’s process scheduler).
- The actual hardware register for the GPIO pin is updated immediately when the kernel processes the write. There is no "partial" or "interrupted" state for the pin.

2.2.5 GPIO Design decision

The GPIO library provides certain multi-task protections, but it has been decided to add a threading locking mechanism on a GPIO.output(). This is a Python built-in locking, and only works within a single python process. In case a system-wide lock is required, we need to use a locking which is file based.

3 Top-down layered static view



The Oradio3 software layering typically consists of three main layers:

- the **hardware abstraction layer** (HAL), which includes the physical components.
- the **middle-ware layer**, which facilitates communication and data management between the hardware and applications. The OS services is considered to be part of this layer
- the **application layer**, where the actual software applications run and perform specific tasks.

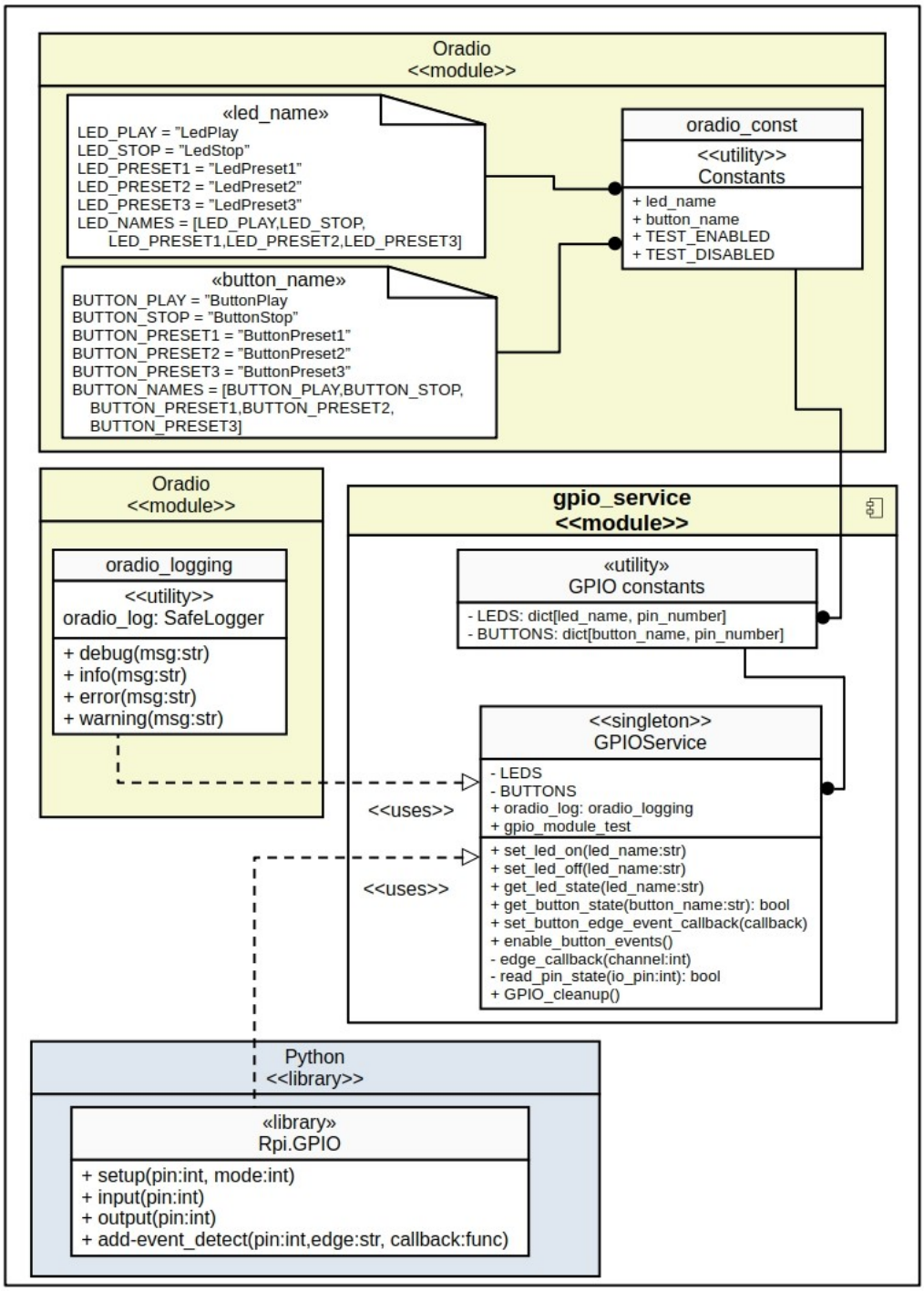
The **Python Layer** is a supporting layer to include Python modules and libraries.
The **Operating System layer** is a supporting layer that provides interaction with the Operating System, e.g. to access devices maintained by the OS.

The GPIOService class is a Oradio3 module offering LED I/O service and BUTTON I/O service to other modules. It includes the Python’s Rpi.GPIO library to read from input pins or write to output pins.

The Rpi.GPIO library abstracts the details of the hardware interactions, providing a simplified API, and relies on RaspiOS for critical functionalities such as threading, memory management, and executing system commands. The RPi.GPIO library uses OS-level calls to toggle GPIO pins, taking advantage of the underlying operating system's capabilities.

The GPIO-controller is managing the physical I/O of the broadcom chip by accessing the specific I/O-registers.

4 Detailed design of the gpio_service module.



4.1 GPIOService Class

4.1.1 Purpose

- Provides a wrapper for button inputs and leds outputs, to hide the actual IO-pinning details
- Provides a callback for a button application to be notified if the state of a button changes
- The output to a pin is atomic, so is non-preemptive.
- Centralizes error handling and logging for I/O error
- The GPIO mode is set to GPIO.BCM

4.1.2 Design & Threading

- **Singleton:** Restricts the class to a single instance, to offer a single point of access for the gpio services.
- **Thread Safety:** Is provided by a python threading.locking on gpio.output(). The Rpi.GPIO library also provides guarantees on kernel level that the write operation completes in one uninterruptible step (atomicity).
- **Events**: The configured button event detection is a kernel-level edge-detection mechanism and uses a background watcher (internal notifier) inside the RPi.GPIO extension to dispatch callbacks.

Design Note:

To prevent certain race-conditions it is important to use the correct order during initialization of the GPIO events.

1) First the method **set_button_edge_event_callback**(*self*, callback) has to set the required callback for the caller.

2) Then the button events can be initialized with **GPIOPIO.add_event_detect(channel, GPIO.RISING, callback=my_callback)**

As soon as the event detection is initialized an edge detection event is found. In case no correct callback (e.g.None) is set, there will be an error, as the callback is not yet available.

4.1.3 API Reference

4.1.3.1 GPIO Constants

LEDS pinout dictionary:

```
LEDS: dict[str, int] = {
    LED_PLAY: 15,
    LED_PRESET1: 24,
    LED_PRESET2: 25,
    LED_PRESET3: 7,
    LED_STOP: 23
}

LED_PLAY    = "LedPlay"
LED_STOP    = "LedStop"
LED_PRESET1 = "LedPreset1"
LED_PRESET2 = "LedPreset2"
LED_PRESET3 = "LedPreset3"
```

BUTTONS pinout dictionary:

```
BUTTONS: dict[str, int] = {
    BUTTON_PLAY: 9,
    BUTTON_PRESET1: 11,
    BUTTON_PRESET2: 5,
    BUTTON_PRESET3: 10,
    BUTTON_STOP: 6,
}

BUTTON_PLAY     = "ButtonPlay"
BUTTON_STOP     = "ButtonStop"
BUTTON_PRESET1  = "ButtonPreset1"
BUTTON_PRESET2  = "ButtonPreset2"
BUTTON_PRESET3  = "ButtonPreset3"
```

4.1.3.2 Methods:

Method	Arguments	Returns
`GPIOService.set_led_on`	`(led_name: str)`	None
Description: • Thread-safe setting ON of a led. • led_name is [`LED_PLAY` | `LED_STOP` | `LED_PRESET1` | `LED_PRESET2` | `LED_PRESET3`]		
Errors: • *"Unknown led name: %s"*, `led_name)`		

Method	Arguments	Returns
`GPIOService.set_led_off`	`(led_name: str)`	None
Description: • Thread-safe setting OFF of a led • led_name is [`LED_PLAY` | `LED_STOP` | `LED_PRESET1` | `LED_PRESET2` | `LED_PRESET3`]		
Errors: • *"Unknown led name: %s"*, `led_name)`		

Method	Arguments	Returns
`GPIOService.get_led_state`	`(led_name: str)`	• True for led ON state, • False for led OFF state • None for unknown led name
Description: • Reads and returns the actual state of a led • led_name is [`LED_PLAY` | `LED_STOP` | `LED_PRESET1` | `LED_PRESET2` |`LED_PRESET3`]		
Errors: • *"Unknown led name: %s"*, `led_name)`		

Method	Arguments	Returns
`GPIOService.get_button_state`	`(button_name: str)`	• True for a button press • False for a button not pressed • None for unknown button name
Description: Reads and returns the actual state of a led. • button_name is [`BUTTON_PLAY` | `BUTTON_STOP` | `BUTTON_PRESET1` | `BUTTON_PRESET2` | `BUTTON_PRESET3`]		

Errors: • "*Unknown button name: %s*", button_name)

Method	Arguments	Returns
GPIOService.enable_button_events	()	• True for a button press • False for a button not pressed • None for unknown button name
Description: Enables the GPIO edge-detection for the button-pins. • The application callback should be initialized before enabling the button events, with **GPIOService.set_button_edge_event_callback(callback)** • All buttons initialized for edge-detection: [BUTTON_PLAY, BUTTON_STOP,BUTTON_PRESET1, BUTTON_PRESET2, BUTTON_PRESET3]		
Errors: • "Cannot enable button events: callback not set"		

Method	Arguments	Returns
GPIOService.set_button_edge_event_callback	(callback)	None
Description: Reads and returns the actual state of a led • button_name is [BUTTON_PLAY | BUTTON_STOP | BUTTON_PRESET1 | BUTTON_PRESET2 | BUTTON_PRESET3]		
Errors: • "Callback function does not exists"		

4.1.4 Error handling

- All errors are logged using oradio_log.error or .warning.
- During instance creation RuntimeError may arise upon GPIO.setup(), these will be reported in the error log.

4.1.5 Attributes for module test

The class provides an attribute <**gpio_module_test** > to enable or disable module test functions within the class. This is used in the method _edge_callback() for performance measurements, to report BUTTON_PRESSED state of the pressed button and the timestamp

4.1.6 Module test

There is a module test program <gpio_service_test.py> that will provide following tests to test the methods of the class:

- All led-pins ON or OFF
- Single led-pin ON or OFF
- Polling the buttons state

- Button event-callback handling
- Unknown led or button error handling